

VIAL-QMKとRP2040で作る！ 磁気キーボード設計

○はじめに

RP2040ZEROで作る！磁気キーボードを元にしたvial-qmkとrp2040を使用して磁気キーボードを設計参考資料になります。

qmk firmwareでは何も知らない状態からkeyboard.jsonとキーマップを作成して基板を配線するだけで自作キーボードを設計することができます。

磁気キーボードも同様に設計を行うことができます。そのための設定箇所等についてのメモになります。

qmkでの自作キーボード設計、ホールセンサやマルチプレクサの動作や制御について軽く調べておくことでスムーズに設計することができます。

またqmk firmwareとrp2040で製作する以上、ポーリングレートがスキャンレートでデッドゾーンの精度が低遅延なハイスパック機は作れません。

○事前準備

quantum\keyboard.cファイルを一部変更する必要があります。

普段使用しているvial-qmkをコピーしkeyboard.cファイルを置き換えたvial-qmk-heプロジェクトを作成します。

<https://github.com/Tsuiha/vial-qmk/blob/he/quantum/keyboard.c>

○必要な設定

config.hファイルを編集し使用するピンやパラメータの設定を行います。

key_layout.cファイルを作成しmatrixの情報を設定します。

その他vial-qmkと同様に設定します。

○ホールセンサの接続モード

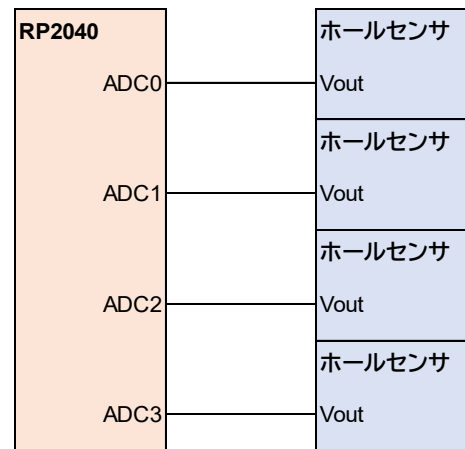
config.hで3パターンの接続モードから一つを有効化します。

```
#define DIRECT_ADC_MODE
#define DIRECT_MUX_MODE
#define CHAIN_MUX_MODE
```

○DIRECT_ADC_MODE

各ADCピンに、一つのホールセンサを接続する方式です。

RP2040に搭載されているADCピンは4つであるため、最大4キーを使用することができます。



○DIRECT_MUX_MODE

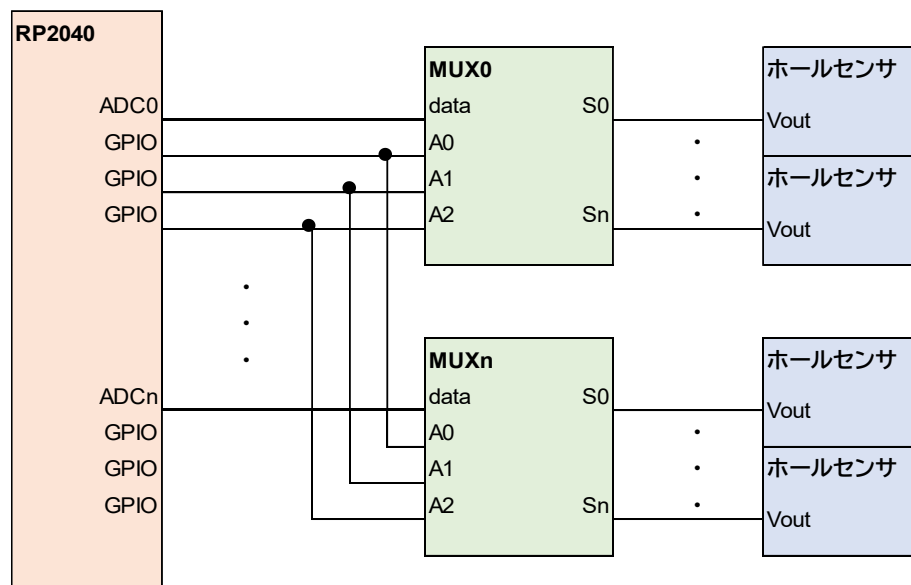
各ADCピンに、一つのMUXを接続する方式です。

RP2040に搭載されているADCピンは4つであるため、最大4つのMUXを使用することができます。

MUXのAddressesピンを制御することでスキャンするホールセンサの選択を行います。

各MUXのAddressセレクトピンはMUX間で共通のGPIOピンで制御します。

8chのMUXを使用した場合最大32キー、16chのMUXを使用した場合最大64キーを使用することができます。



○CHAIN_MUX_MODE

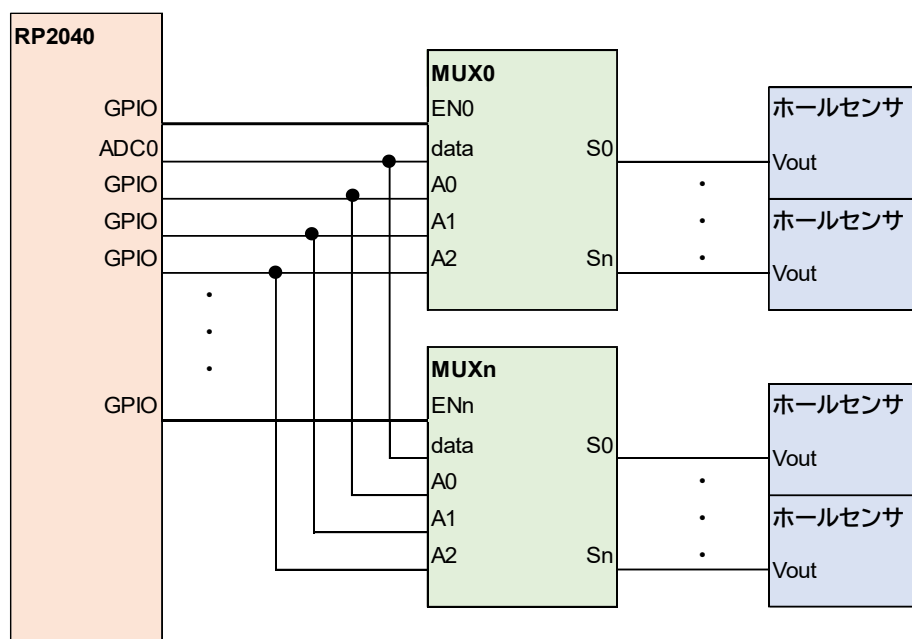
一つのADCピンに、複数のMUXを接続する方式です。

一つのADCピンにすべてのMUXを接続するため、使用するRP2040のADCピンは一つです。

MUXのENピンを制御し有効化/無効化することでスキャンするMUXの選択を行います。

各MUXのデータおよびAddressセレクト端子はMUX間で共通のGPIOピンで制御します。

ENピン用のGPIOを引き出せる限りMUXを接続しキー数を増やすことができます。



○ADC_PINS

ホールセンサやMUXに接続するADCピンを指定します。

DIRECT_ADC_MODE, DIRECT_MUX_MODEの場合は、使用するADCピンを指定してください。

要素数は使用するADCピンの数に対応します。

例) `#define ADC_PINS {GP26, GP27, GP28, GP29}`

ADC_PINSの並び順がmatrixのrowの数および順番に対応しています。

CHAIN_MUX_MODEの場合は、1つの使用するADCピンを指定してください。

例) `#define ADC_PINS {GP26}`

○MUX_ENABLE_PINS

MUXのENピンに接続するGPIOピンを指定します。

DIRECT_ADC_MODE, DIRECT_MUX_MODEの場合は不要なためコメントアウトしてください。

CHAIN_MUX_MODEの場合は、各MUXのENピンに接続するGPIOピンを指定してください。

要素数は使用するMUXの数に対応します。

例) `#define MUX_ENABLE_PINS {GP1, GP2, GP3, GP4, GP5, GP6}`

MUX_ENABLE_PINSの並び順はmatrixのrowの数および順番に対応しています。

○MUX_CH_ORDER

MUXの使用するchを指定します。

DIRECT_ADC_MODEの場合は不要なためコメントアウトしてください。

DIRECT_MUX_MODE, CHAIN_MUX_MODEの場合は、スキャンする必要のあるchを指定してください。

どのMUXでも使用していないchがある場合、指定せず省いてください。

例) #define MUX_CH_ORDER {0, 1, 2, 3, 4, 5, 6, 7}

MUX_CH_ORDERの並び順はmatrixのcolの数および順番に対応しています。

○MUX_SELECT_PINS

MUXのAddressセレクトピンに接続するGPIOピンを指定します。

DIRECT_ADC_MODEの場合は不要なためコメントアウトしてください。

DIRECT_MUX_MODE, CHAIN_MUX_MODEの場合は、セレクトピンに接続するGPIOピンを指定してください。

要素数は使用するMUXのch数の2進数の桁数に対応します。

例) #define MUX_SELECT_PINS {GP1, GP2, GP3}

MUX_SELECT_PINSの並び順は左側がLSB, 右側がMSBに対応しています。

○MATRIX_ROWS

DIRECT_ADC_MODE, DIRECT_MUX_MODEの場合、使用するADCピンの数と対応します。

CHAIN_MUX_MODEの場合は、使用するMUXの数と対応します。

○MATRIX_COLS

DIRECT_ADC_MODEの場合、1になります。

DIRECT_MUX_MODE, CHAIN_MUX_MODEの場合は、使用するMUXのch数と対応します。

○PCBキャリブレーション設定

本機ではPCBキャリブレーション用のスイッチとしてタクトスイッチを使用しており、押した状態で起動するとPCBキャリブレーションが実施されるようになっています。

PCBキャリブレーション用のスイッチに接続するGPIOピンを指定してください。

例) `#define PCB_CAL_PIN GP1`

ホールセンサの磁束密度ゼロ点出力を取得するものです。

使用するホールセンサに合わせた初期値を設定してしまい、PCBキャリブレーション機能をオミットしても使用に問題はありません

(例えばDRV5055であれば2048を、DRV5056であれば744を固定値として設定してしまう)

例) `#define DEF_REF_ADC_VAL 2048`

○スイッチキャリブレーション設定

本機ではキー入力とスイッチキャリブレーションの動作モード切り替えにspdtスライドスイッチを使用しています。

スライドスイッチのスイッチキャリブレーションモード側接点に接続するGPIOピンを指定してください。

例) `#define SW_CAL_PIN GP1`

○感度オフセット設定

本機では入力感度の調整にsp5tスライドスイッチを使用しており、切り替えることで5段階のキー感度を利用することができます。

使用するスライドスイッチの各接点に接続するGPIOピンを指定してください。

例) `#define SENSI_OFFSET_PINS {GP1, GP2, GP3, GP4, GP5}`

要素数は使用するスライドスイッチの接点数に対応します。

各感度ステージでの感度オフセット量をum単位で設定することができます。

例) `#define DEF_OFFSET_PONT {0, 50, 100, 200, 500}`

○SOCD設定

本機ではWASDキーに付与するSOCD機能を3つのモードから選択することができます。
SOCDモードの切り替えにsp3tスライドスイッチを使用しています。各端子に接続するGPIOピンを指定する必要があります。

例) #define SOCD_PINS {GP6, GP7, GP8}

SOCD_PINSの並び順は左からOFF, HYBRID, FORCEDに対応しています。

SOCDを使用しない場合はコメントアウトしてください。

HYBRIDモードで、ボトムアウトから何umの範囲を底打ちエリアとするか設定することができます。

例) #define DEF_CCL_TH 100

○ゲーミングキー設定

本機ではゲームでの使用を想定し、WASDキーを指定することで高感度入力およびSOCD機能を付与することができます。

使用しない場合はコメントアウトしてください。

各キーのindex番号を指定する必要があります。

$idx = row * MATRIX_COLS + col$ で求めることができます。

例)

#define W_KEY_IDX 13

#define A_KEY_IDX 10

#define S_KEY_IDX 12

#define D_KEY_IDX 14

○スイッチストローク

使用するスイッチのストロークを指定してください。

#define DEF_STROKE 3400

本機の使用ではスイッチストロークの異なるスイッチを使用した場合、感度のみ変化します。

設定ストローク：設定感度 = 実ストローク：実感度

○KLE to LAYOUT

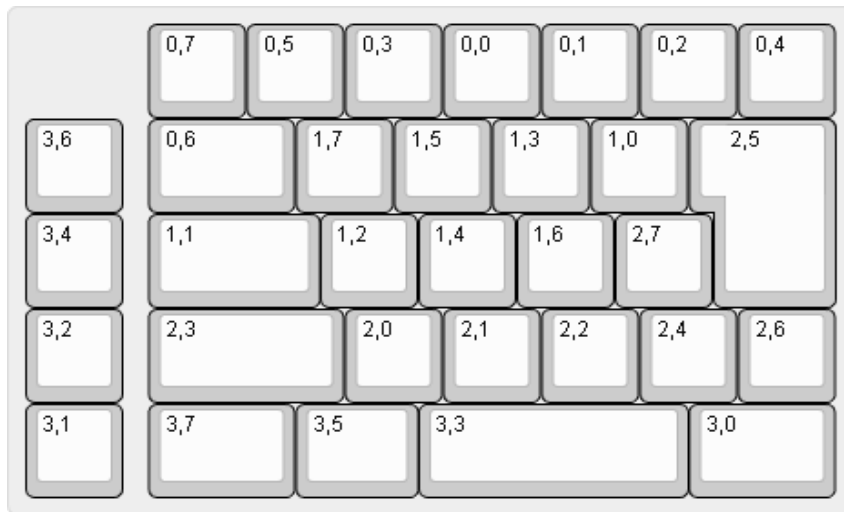
key_layout.cというファイルを作成し、kbd直下に格納する必要があります。

Key_layout.clはmatrix内各キーの有効無効の指定と、座標等のレイアウト情報を保持しています。

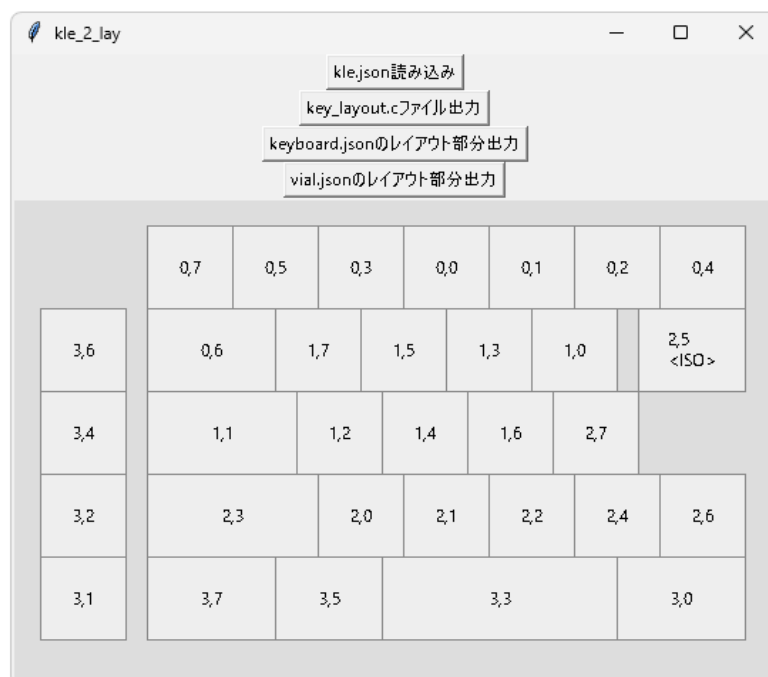
$idx = row * MATRIX_COLS + col$ として、各キーの有効無効情報をidx順に並べています。

手打ちしてもよいですが、KLEで出力したjsonファイルから変換するpywスクリプトが用意されています。

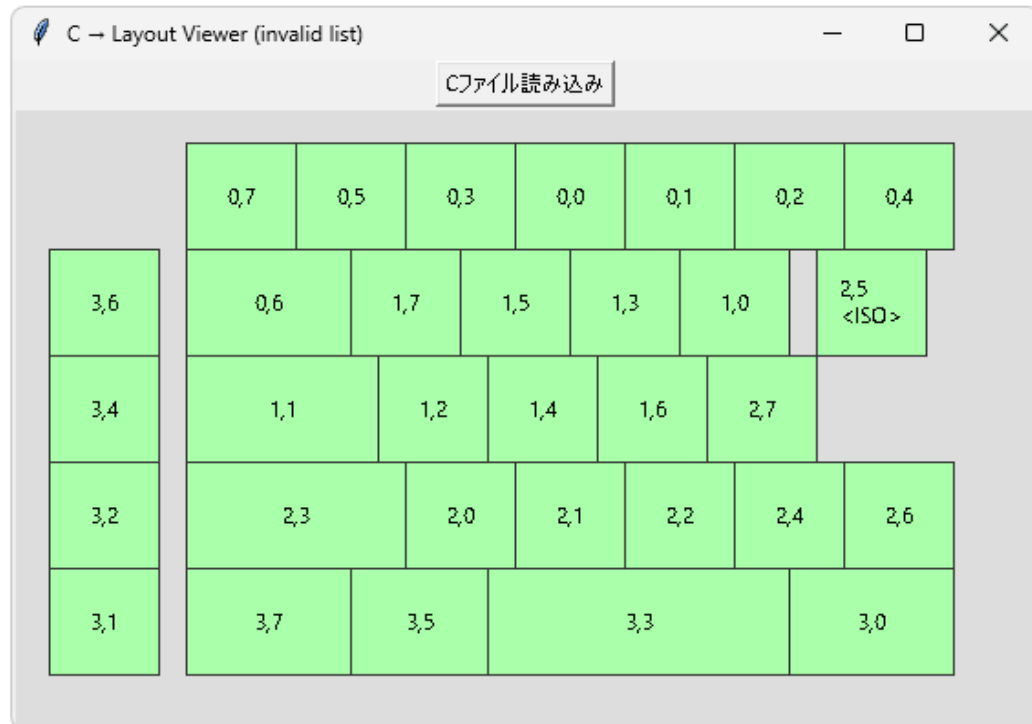
普段vial.json用のlayoutデータを作成するのと同様にKLEで作成したレイアウトのlegendに row,col を入力し、keyboard-layout.jsonファイルを出力します。



kle_2_lay.pyw を起動し、kle.json読み込みボタンからkeyboard-layout.jsonファイルを読み込みます。レイアウトのプレビューが生成されますので問題ないか確認し、各種ファイルの出力ボタンを押します。key_layout.c・・・出力されたものをkbd直下に格納してください。keyboard.jsonのレイアウト部分・・・出力データをそのままkeyboard.json該当部に使用できます。vial.jsonのレイアウト部分・・・出力データをそのままvial.json該当部に使用できます。

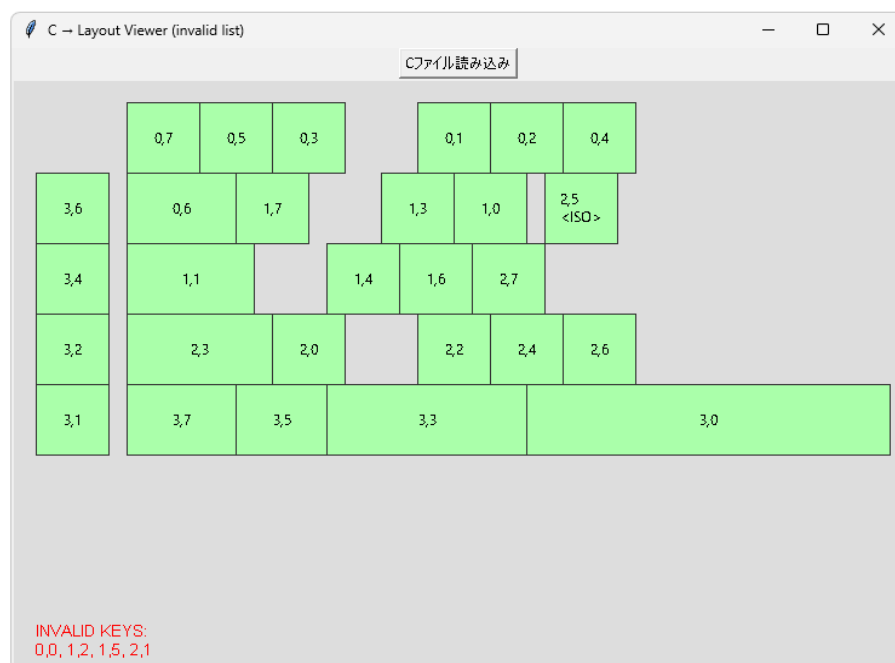


c_2_lay.pywを使用することで、kay_layout.cからレイアウト情報を復元することができます。



本機ではMUXのchをすべて使用していますが、matrix内に使用しない(row,col)がある場合の方が多いと思います。

そうしたレイアウトを読み込んだ場合、下部にinvalid keysの一覧が表示されます。



○部品選び

本機はKS-20互換スイッチとDRV5055のようなスイッチを押すとホールセンサの出力電圧が上昇する組み合わせに対応しています。

電子部品に関してはデータシートがしっかりしているベンダーの製品がいいと思います。

・ホールセンサ

DRV5055シリーズ・・・消費電力が比較的小さい

1mm, 1.2mm pcbの場合DRV5055A3

1.6mm pcbの場合DRV5055A2

MT910Xシリーズ・・・ちょっと安い

1mm, 1.2mm pcbの場合MT9102

1.6mm pcbの場合MT9105

1.2mm pcbかつ最近増えている高磁束スイッチとやらを使用しない場合MT9103

候補は他にもありますが、入手性を考えるとこのあたりがおすすめです。やや高価ですが私はDRV5055しか使わないと思います。

・MUX

厳密な話をすると変わるのでしょうが大した差はないため、まともそうで入手性の良いものを選択すればよいと思います。

私は特に理由はないですがTMUX1308を使用しています。

・磁気スイッチ

一般的であるためKS-20互換と呼ばれているものを使用します。

平均的な磁気スイッチの軸ブレによる磁束密度変動量をストロークの変動量に換算すると大体0.05mm程度となります。酷いものだと換算0.1mm程度ブレますし、優秀なものでも換算0.02mm程度はブレるものです。

トップアウト状態でキーキャップをなでたり、ボトムアウト状態でグリグリしたりすることでそれだけ検出ストロークがブレるということです。

そのため軸ブレの小さなスイッチを使用することが精度の面では望ましく、新しめの製品の方が基本的にはよいです。

ショートストロークが好みの場合、ZENAIMスイッチをKS-20互換スイッチ向けと同じ基板で使うことができます。プレートは別途設計が必要です。